Simularea mișcării corpurilor cerești în două dimensiuni

Model matematic, implementare și verificarea rezultatelor

Georgescu Andrei

29.10.2026

1 Introducere

Proiectul simulează mișcarea unor corpuri care se atrag gravitațional. Fiecare corp este descris prin masă, rază, poziție și viteză. Pornind de la o configurație inițială, programul calculează forțele dintre corpuri și urmărește evoluția lor în timp. Dacă două corpuri ajung în contact, ele sunt unite într-un singur corp.

Calculul este realizat în C++, iar rezultatele sunt salvate într-un fișier binar. Un program Python citește acest fișier și afișează traiectoriile. Astfel, calculul mișcării și vizualizarea rezultatelor sunt două etape separate.

Documentația prezintă mai întâi formulele fizice și modul în care au fost implementate. Urmează metoda de integrare, organizarea programului și formatul datelor. Ultima parte este dedicată exemplelor: mișcarea a două corpuri, un sistem cu trei corpuri și o coliziune între patru corpuri.

Simularea cu două corpuri este folosită pentru verificarea preciziei, deoarece mișcarea poate fi calculată și prin soluția analitică a problemei. Comparația urmărește separat conservarea energiei și diferența dintre pozițiile numerice și cele analitice. Celelalte exemple arată comportamentul programului atunci când apar interacțiuni suplimentare sau coliziuni.

2 Modelul fizic și implementarea formulelor

2.1 Mărimile asociate unui corp

Pentru corpul i se folosesc masa m_i, raza R_i, poziția $\mathbf{r}_i = (x_i, y_i)$, viteza $\mathbf{v}_i = (v_{x,i}, v_{y,i})$ și accelerația $\mathbf{a}_i = (a_{x,i}, a_{y,i})$. Toate mărimile sunt exprimate în unități SI: kilograme, metri și secunde.

În cod, componentele unui vector sunt păstrate în clasa `vector_2d`. Clasa `celestial_object` conține proprietățile unui corp. Funcțiile de acces, precum `get_position()` și `get_velocity()`, permit citirea stării, iar funcțiile `set_position()` și `set_velocity()` o actualizează.

Mișcarea are loc într-un plan, însă forța folosită este atracția gravitațională newtoniană, proporțională cu $1/r^2$. Razele sunt folosite la verificarea contactului dintre corpuri.

2.2 Calculul forței gravitaționale

Pentru două corpuri, deplasarea dintre centre și distanța sunt

$$\mathbf{d}_{ij} = \mathbf{r}_j - \mathbf{r}_i = (x_j - x_i,\ y_j - y_i), \qquad r_{ij} = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}. \tag{1}$$

Modulul forței este dat de legea atracției universale:

$$F_{ij} = G\frac{m_i m_j}{r_{ij}^2}, \qquad G = 6{,}67430 \times 10^{-11}\ \mathrm{m^3\,kg^{-1}\,s^{-2}}. \tag{2}$$

Pentru a obține și direcția, forța este înmulțită cu vectorul unitar orientat de la corpul i spre corpul j:

$$\mathbf{F}_{ij} = G m_i m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{r_{ij}^3}. \tag{3}$$

Prin urmare, componentele calculate de program sunt

$$F_{x,ij} = F_{ij}\frac{x_j - x_i}{r_{ij}}, \qquad F_{y,ij} = F_{ij}\frac{y_j - y_i}{r_{ij}}. \tag{4}$$

Implementarea din `set_force` urmează direct aceste relații:

Listarea 1: Calculul și aplicarea forței pentru o pereche de corpuri.

```
const vector_2d displacement(b.x - a.x, b.y - a.y);
const double distance =
    std::hypot(displacement.x, displacement.y);
const double magnitude =
    G * objects[i].get_mass() * objects[j].get_mass()
    / distance / distance;
const vector_2d force =
    displacement * (magnitude / distance);

forces[i] = forces[i] + force;
forces[j] = forces[j] + force * (-1.0);
```

Forța asupra corpului j este opusă celei asupra corpului i. Fiecare pereche este calculată o singură dată, în buclele cu $i < j$. Forța totală asupra unui corp este suma contribuțiilor tuturor celorlalte corpuri:

$$\mathbf{F}_i = \sum_{j \neq i} \mathbf{F}_{ij}, \qquad \mathbf{a}_i = \frac{\mathbf{F}_i}{m_i}. \tag{5}$$

Pentru N corpuri sunt calculate $N(N-1)/2$ perechi la fiecare evaluare a forțelor.

2.3 Calculul energiei

Energia mecanică totală este suma energiei cinetice și a energiei potențiale gravitaționale:

$$K = \sum_i \frac{m_i}{2} \left(v_{x,i}^2 + v_{y,i}^2 \right), \tag{6}$$

$$U = -\sum_{i<j} \frac{G m_i m_j}{r_{ij}}, \tag{7}$$

$$E = K + U. \tag{8}$$

Energia potențială este negativă deoarece valoarea zero este aleasă la distanță infinită. Perechile apar o singură dată în sumă, pentru a nu dubla energia potențială.

Funcția `total_energy` adună energia cinetică a fiecărui corp și scade potențialul fiecărei perechi. Fragmentul următor conține termenii principali; în funcție, fiecare este plasat în bucla corespunzătoare:

Listarea 2: Termenii folosiți în calculul energiei totale.

```
energy += 0.5 * objects[i].get_mass()
        * (velocity.x * velocity.x + velocity.y * velocity.y);

energy -= G * objects[i].get_mass()
            * objects[j].get_mass() / distance;
```

În absența coliziunilor, energia unui sistem izolat ar trebui să rămână constantă. Diferența față de energia inițială este folosită în exemplele următoare pentru a măsura eroarea numerică.

2.4 Detectarea coliziunilor și unirea corpurilor

Două corpuri sunt considerate în contact dacă

$$r_{AB} \leq R_A + R_B. \tag{9}$$

La contact, ele sunt înlocuite cu un singur corp. Masa nouă este suma maselor, iar poziția și viteza sunt medii ponderate cu masele:

$$M = m_A + m_B, \tag{10}$$

$$\mathbf{R} = \frac{m_A \mathbf{r}_A + m_B \mathbf{r}_B}{M}, \tag{11}$$

$$\mathbf{V} = \frac{m_A \mathbf{v}_A + m_B \mathbf{v}_B}{M}. \tag{12}$$

Această alegere conservă masa și impulsul liniar. Raza nouă este calculată prin adunarea volumelor sferice:

$$R_{\text{nou}} = \sqrt[3]{R_A^3 + R_B^3}. \tag{13}$$

În `merge_celestial_objects`, corpul rezultat înlocuiește unul dintre cele două elemente, celălalt este eliminat, iar numărul de corpuri active scade. Funcția `resolve_collisions` reia verificarea după fiecare unire, deoarece corpul rezultat poate intra în contact și cu un alt corp.

Coliziunea este modelată ca perfect inelastică. Energia cinetică de translație se modifică prin

$$\Delta K = -\frac{1}{2}\frac{m_A m_B}{m_A + m_B}\|\mathbf{v}_A - \mathbf{v}_B\|^2. \tag{14}$$

În plus, din energia calculată dispare potențialul perechii unite. Programul nu urmărește energia internă a corpului rezultat. De aceea, variația energiei în timpul unei coliziuni are o interpretare diferită de eroarea unei orbite fără contact.

3 Metoda numerică și organizarea programului

3.1 Integrarea mișcării prin metoda leapfrog

Ecuațiile mișcării sunt $\dot{\mathbf{r}}_i = \mathbf{v}_i$ și $\dot{\mathbf{v}}_i = \mathbf{a}_i$. Programul le integrează folosind un pas temporal fix h. Un pas complet are trei etape:

$$\mathbf{v}_i^{n+1/2} = \mathbf{v}_i^n + \frac{h}{2}\mathbf{a}_i(\mathbf{r}^n), \tag{15}$$

$$\mathbf{r}_i^{n+1} = \mathbf{r}_i^n + h\mathbf{v}_i^{n+1/2}, \tag{16}$$

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^{n+1/2} + \frac{h}{2}\mathbf{a}_i(\mathbf{r}^{n+1}). \tag{17}$$

Mai întâi se actualizează viteza pe jumătate de pas. Apoi se deplasează corpul pe un pas întreg, folosind viteza intermediară. Forțele se recalculează la pozițiile noi, după care se completează a doua jumătate de pas a vitezei. Această variantă a metodei leapfrog este echivalentă cu velocity Verlet pentru forțe care depind de poziție [1].

Funcția `update_velocity` aplică relația $\mathbf{v} \leftarrow \mathbf{v} + (\mathbf{F}/m)\Delta t$, iar `update_position` aplică $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{v}\Delta t$. În programul principal, ordinea apelurilor este:

Listarea 3: Succesiunea actualizărilor într-un pas de integrare.

```
// Fortele sunt cunoscute la pozitiile curente.
for (int j = 0; j < number_of_celestial_objects; j++)
    update_velocity(forces[j], celestial_objects[j], time_step / 2);

for (int j = 0; j < number_of_celestial_objects; j++)
    update_position(celestial_objects[j], time_step);

set_force(forces, celestial_objects, number_of_celestial_objects);

for (int j = 0; j < number_of_celestial_objects; j++)
    update_velocity(forces[j], celestial_objects[j], time_step / 2);
```

Toate pozițiile sunt actualizate înainte de recalcularea forțelor. Astfel, interacțiunile sunt evaluate pentru o singură configurație a sistemului, la același moment.

Pentru o mișcare fără coliziuni, metoda are eroare globală de ordinul doi: reducerea pasului de două ori conduce, în regimul în care eroarea de discretizare domină, la o reducere de aproximativ patru ori a erorii. Metoda nu conservă energia exact la fiecare pas; în exemplele orbitale, abaterea energetică oscilează în jurul valorii inițiale.

3.2 Citirea condițiilor inițiale

Fișierul `input.txt` începe cu numărul de corpuri N, durata simulării T și pasul temporal h. Fiecare linie următoare descrie un corp, în ordinea

$$m, \quad R, \quad x, \quad y, \quad v_x, \quad v_y. \tag{18}$$

Programul citește corpurile, calculează energia inițială și intră în bucla temporală. La fiecare pas sunt evaluate energia și starea curentă, sunt salvate pozițiile, apoi sunt aplicate actualizările mișcării. Toate corpurile active sunt integrate,

inclusiv cel mai masiv.

3.3 Salvarea și vizualizarea rezultatelor

Fișierul `simulation.bin` conține mai întâi numărul inițial de corpuri, durata și pasul temporal. Pentru fiecare stare se salvează timpul, numărul de corpuri active și coordonatele (x, y) ale fiecărui corp. În formatul folosit, antetul ocupă 20 de octeți, iar o stare cu N corpuri ocupă $12 + 16N$ octeți.

În Python, antetul și începutul fiecărei stări sunt citite astfel:

```
count, duration, step = struct.unpack('<idd', header)
time, active = struct.unpack('<di', frame_header)
```

Coordonatele sunt transformate în tablouri NumPy. Vitezele și masele nu sunt stocate în acest format, astfel încât analiza folosește și condițiile inițiale.

Programul `graph.py` afișează traseele și pozițiile corpurilor, cu aceeași scară pe ambele axe. Vizualizarea poate fi oprită sau reluată, deplasată cu un pas sau adusă la un anumit timp. După o coliziune, traseele sunt reluate pentru noul număr de corpuri, deoarece indicii obiectelor se pot schimba. Aceste comenzi controlează redarea rezultatului deja calculat.

4 Exemplul 1: două corpuri comparate cu soluția analitică

4.1 Configurația simulată

Acest exemplu verifică dacă programul reproduce mișcarea orbitală a două corpuri izolate. Masele sunt diferite, iar ambele corpuri se deplasează în jurul centrului comun de masă. Simularea durează două zile, adică 172800 s.

```
2 172800 0.1
1e15 100 -5000 0 0 -0.65
5e14 80 10000 0 0 1.30
```

Primul corp are masa 10^{15} kg și pornește din $(-5000, 0)$ m cu viteza $(0, -0{,}65)$ m/s. Al doilea are masa 5×10^{14} kg, poziția inițială $(10000, 0)$ m și viteza $(0, 1{,}30)$ m/s. Separarea inițială este 15 km. Impulsul total este nul, deci centrul de masă rămâne în origine.

Aceleași condiții au fost simulate cu pașii de 5 s, 2 s, 1 s, 0,5 s și 0,1 s. Pentru comparațiile de mai jos, rulările au fost refăcute folosind funcțiile C++ ale proiectului. Energia a fost măsurată la fiecare pas, iar pozițiile au fost comparate la intervale de 4–5 s, incluzând ultima stare. Rezultatele de poziție prezentate sunt maxime în punctele verificate.

4.2 Calculul soluției analitice

Problema este simplificată prin folosirea poziției și vitezei relative:

$$\mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1, \qquad \mathbf{v} = \mathbf{v}_2 - \mathbf{v}_1, \qquad \mu = G(m_1 + m_2). \tag{19}$$

Mișcarea relativă satisface ecuația $\ddot{\mathbf{r}} = -\mu\mathbf{r}/r^3$. Pentru condițiile inițiale de mai sus, orbita este o elipsă. Semiaxa mare a și excentricitatea e se obțin din energia specifică și momentul cinetic specific:

$$\varepsilon = \frac{v^2}{2} - \frac{\mu}{r}, \qquad a = -\frac{\mu}{2\varepsilon}, \qquad e = \sqrt{1 + \frac{2\varepsilon\|\mathbf{r} \times \mathbf{v}\|^2}{\mu^2}}. \tag{20}$$

Rezultă $a = 10487{,}476561$ m și $e = 0{,}4302773325$. Distanța minimă dintre centre este $5974{,}953122$ m, iar cea maximă este 15000 m. Perioada orbitală este

$$P = 2\pi\sqrt{\frac{a^3}{\mu}} = 21327{,}398405 \text{ s}. \tag{21}$$

În cele două zile sunt parcurse aproximativ 8,1 perioade. Distanța minimă este mult mai mare decât suma razelor, de 180 m, deci acest exemplu nu implică o coliziune.

Pentru calculul poziției la un timp t se folosește ecuația Kepler [2]:

$$u - e\sin u = M(t), \qquad M(t) = \pi + \sqrt{\frac{\mu}{a^3}}\, t. \tag{22}$$

Mărimea u este anomalia excentrică, un unghi care descrie poziția pe elipsă. Ea se obține prin rezolvarea ecuației scalare de mai sus. În orientarea acestui exemplu, coordonatele relative sunt

$$x(t) = -a(\cos u - e), \qquad y(t) = -a\sqrt{1-e^2}\,\sin u. \tag{23}$$

Pozițiile celor două corpuri rezultă prin împărțirea vectorului relativ în raport invers cu masele:

$$\mathbf{r}_1 = -\frac{m_2}{m_1+m_2}\mathbf{r}, \qquad \mathbf{r}_2 = \frac{m_1}{m_1+m_2}\mathbf{r}. \tag{24}$$

Pozițiile analitice sunt evaluate la aceiași timpi ca pozițiile numerice. În acest fel, diferența măsurată nu provine din compararea unor momente diferite.

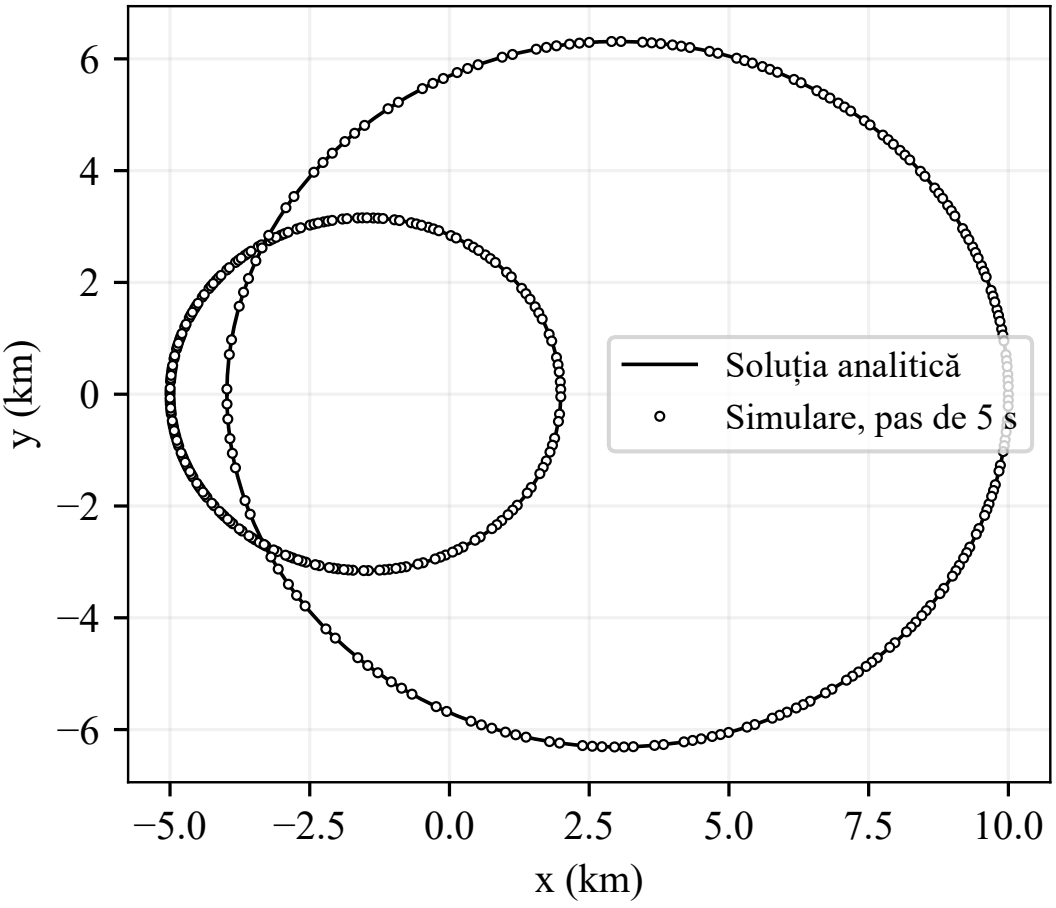


Figura 1: Traiectoriile celor două corpuri. Linia continuă reprezintă soluția analitică, iar cercurile reprezintă poziții din simularea cu pas de 5 s. La scara figurii, traseele aproape se suprapun; eroarea este măsurată separat în continuare.

4.3 Eroarea energiei

Energia inițială a sistemului este $E_0 = -1{,}5910166667 \times 10^{15}$ J. Pentru fiecare stare se calculează abaterea

$$\Delta E(t) = E(t) - E_0. \tag{25}$$

Eroarea maximă absolută și eroarea relativă maximă sunt

$$\Delta E_{\max} = \max_t |E(t) - E_0|, \qquad \epsilon_E = \frac{\Delta E_{\max}}{|E_0|}. \tag{26}$$

Raportarea la $|E_0|$ permite interpretarea unei erori exprimate în jouli în raport cu energia totală a sistemului.

Pasul h (s)	Eroarea maximă (J)	Eroarea relativă maximă
5	$5{,}071 \times 10^{9}$	$3{,}187 \times 10^{-6}$
2	$8{,}113 \times 10^{8}$	$5{,}100 \times 10^{-7}$
1	$2{,}028 \times 10^{8}$	$1{,}275 \times 10^{-7}$
0,5	$5{,}071 \times 10^{7}$	$3{,}187 \times 10^{-8}$
0,1	$2{,}028 \times 10^{6}$	$1{,}275 \times 10^{-9}$

Pentru pasul de 5 s, eroarea maximă este aproximativ $5{,}071 \times 10^{9}$ J, adică $0{,}0003187\%$ din modulul energiei inițiale. La pasul de 0,1 s, ea scade la aproximativ $2{,}028 \times 10^{6}$ J, adică $0{,}0000001275\%$. Reducerea erorii este de aproximativ 2500 de ori.

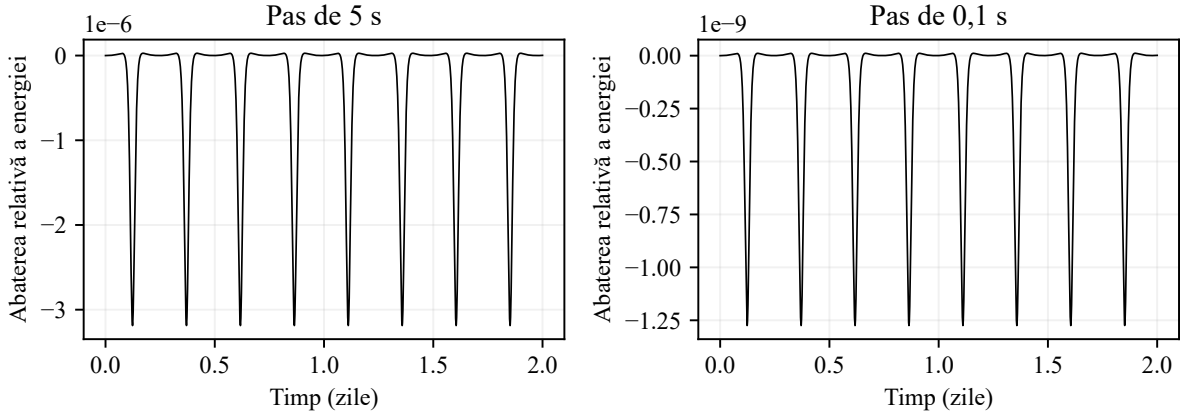


Figura 2: Abaterea relativă a energiei, $(E(t) - E_0)/|E_0|$, pentru pașii de 5 s și 0,1 s. Axele verticale au scări diferite pentru ca oscilațiile ambelor rulări să fie vizibile.

În afișarea programului există două valori diferite. `energy error` este $E_{\max} - E_0$ și arată cea mai mare creștere a energiei. `max energy error` este $\max |E(t) - E_0|$ și include atât creșterile, cât și scăderile. Pentru această orbită, cea mai mare abatere este o scădere; de aceea cele două numere nu sunt egale.

Valorile comunicate pentru cele două rulări inițiale au fost $5\,070\,864\,995{,}75$ J și $2\,028\,292{,}25$ J pentru eroarea absolută maximă. Rulările refăcute cu pașii de 5 s și 0,1 s dau rezultate foarte apropiate. Tabelele și graficele de aici folosesc rulările refăcute, pentru care sunt cunoscute explicit pasul și pozițiile comparate.

4.4 Eroarea traiectoriei

Pentru fiecare corp, eroarea poziției este distanța dintre poziția simulată și poziția analitică la același timp:

$$\delta_i(t) = \|\mathbf{r}_{i,\text{sim}}(t) - \mathbf{r}_{i,\text{analitic}}(t)\|. \tag{27}$$

Se calculează și eroarea vectorului de separare dintre corpuri:

$$\delta_{\text{rel}}(t) = \|[\mathbf{r}_{2,\text{sim}} - \mathbf{r}_{1,\text{sim}}] - [\mathbf{r}_{2,\text{analitic}} - \mathbf{r}_{1,\text{analitic}}]\|. \tag{28}$$

Această eroare arată cât diferă mișcarea unui corp față de celălalt. În configurația studiată, eroarea primului corp este o treime din eroarea relativă, iar cea a celui de-al doilea este două treimi.

Pasul h (s)	Corpul 1 (mm)	Corpul 2 (mm)	Separarea (mm)
5	251,006	502,011	753,017
2	40,161	80,322	120,483
1	10,040	20,081	30,121
0,5	2,510	5,020	7,530
0,1	0,104	0,208	0,312

Tabelul prezintă erorile maxime în pozițiile verificate. La pasul de 5 s, eroarea maximă a separării este 753 mm. La pasul de 0,1 s, ea este 0,312 mm, iar erorile individuale sunt 0,104 mm și 0,208 mm. La ultimul timp salvat, eroarea separării este 0,302 mm pentru pasul de 0,1 s.

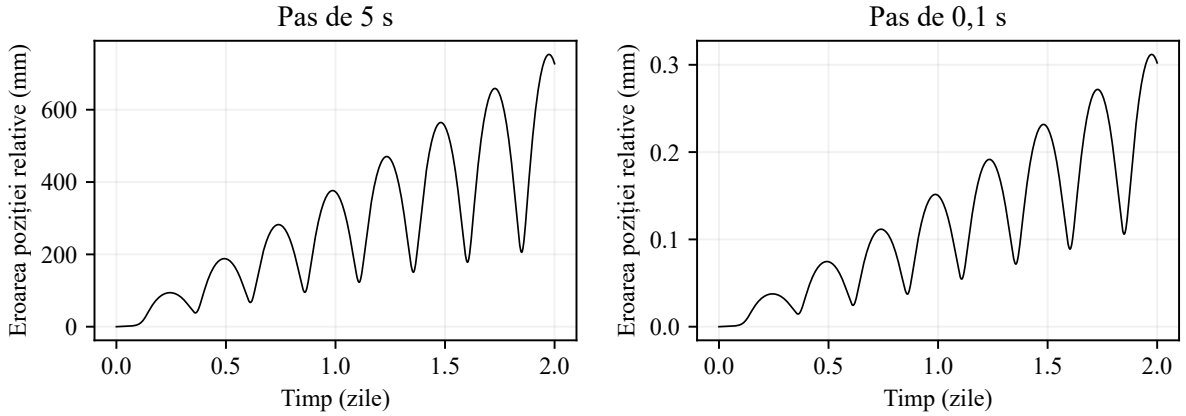


Figura 3: Eroarea poziției relative față de soluția analitică, în milimetri. Fiecare rulare are propria scară verticală. Variația în timp include atât diferențele de formă ale orbitei, cât și diferențele de poziție de-a lungul ei.

4.5 Efectul reducerii pasului temporal

Datele arată că reducerea pasului îmbunătățește atât energia, cât și traiectoria. Pentru pașii de 1 s și 0,5 s, eroarea maximă de poziție relativă scade de la aproximativ 30,12 mm la 7,53 mm: o reducere de patru ori la înjumătățirea pasului.

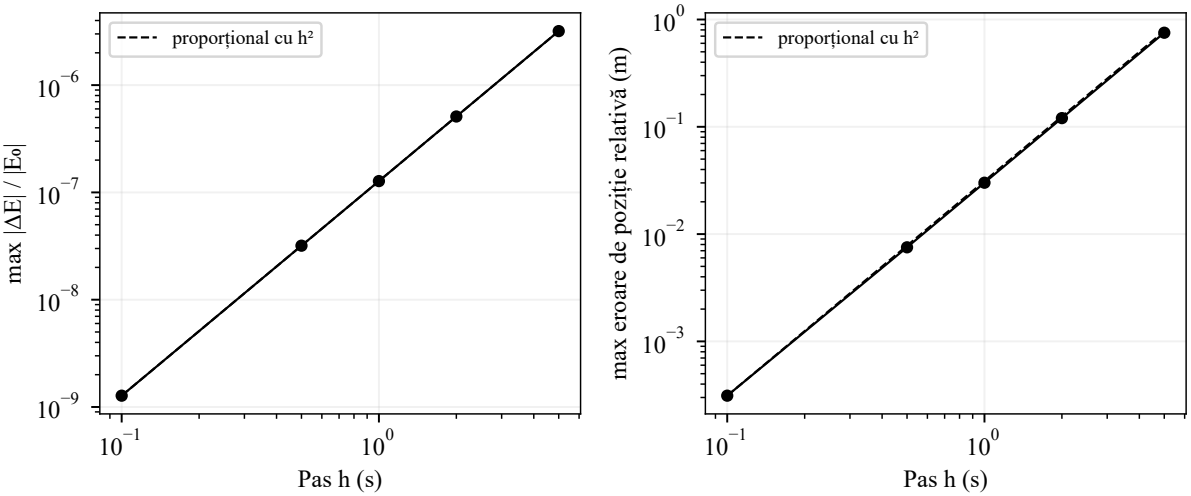


Figura 4: Dependența erorilor de pasul temporal. Linia întreruptă arată o variație proporțională cu h^2. Punctele sunt erorile măsurate în cele cinci rulări.

O ajustare a relației eroare $\approx Ch^p$ dă $p \approx 2{,}000$ pentru energie și $p \approx 1{,}991$ pentru poziție. Rezultatele sunt în acord cu ordinul doi al metodei leapfrog.

În rularea cu pas de 0,1 s, ultimul timp salvat este aproximativ $172799{,}90000557$ s, din cauza adunărilor repetate ale pasului în reprezentarea `double`. Comparația analitică folosește acest timp efectiv. Soluția analitică a fost verificată și printr-un calcul DOP853 independent; la toleranța relativă de 10^{-13}, diferența maximă a fost aproximativ 0,0013 mm, sub eroarea de poziție măsurată.

Rezultatul acestui exemplu este, așadar, o verificare cantitativă a programului pentru o orbită eliptică: la pasul de 0,1 s, erorile individuale maxime în punctele verificate sunt sub 0,21 mm pe un interval de două zile.

5 Exemplul 2: mișcarea a trei corpuri

5.1 Configurația simulată

În acest exemplu, două corpuri mai mici se deplasează în jurul unui corp mult mai masiv. Toate cele trei corpuri se atrag reciproc și sunt actualizate de program. Durata este de două zile, iar pasul temporal este de 0,5 s.

```
3 172800 0.5
1e15 100 0 0 0 0.000016
1e10 10 10000 0 0 2.0
3e10 15 -30000 0 0 -1.2
```

Masele sunt 10^{15} kg, 10^{10} kg și 3×10^{10} kg. Corpul interior pornește la 10 km de origine cu viteza de 2 m/s, iar cel exterior la 30 km, cu viteza de 1,2 m/s în sens opus. Viteza inițială a corpului masiv este aleasă astfel încât impulsul total să fie nul.

5.2 Traiectoriile și comparația cu un calcul independent

Sistemul general cu trei corpuri nu are soluția analitică simplă folosită în exemplul anterior. Pentru verificare s-a folosit DOP853, o metodă numerică de ordin înalt disponibilă în SciPy [3]. Acest calcul integrează interacțiunile dintre toate cele trei corpuri.

Au fost făcute două calcule de referință, cu toleranțe relative de 10^{-12} și 3×10^{-14}. Diferența maximă dintre ele a fost 0,0312 mm. Pozițiile din simulare au fost comparate cu referința mai precisă la 17281 de timpi, din 10 în 10 s, incluzând ultima stare.

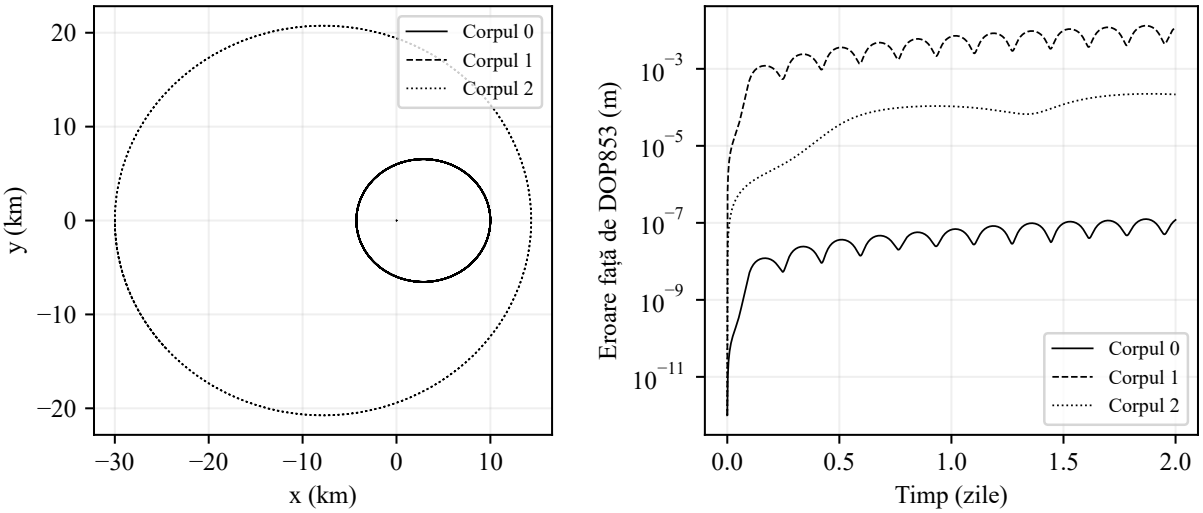


Figura 5: Simularea cu trei corpuri. Stânga: traiectoriile. Dreapta: diferența pozițiilor față de calculul DOP853, la aceiași timpi. Indicii 0, 1 și 2 urmează ordinea corpurilor din fișierul de intrare.

Corp	Eroarea maximă (mm)	Eroarea la final (mm)
Corpul masiv	0,000125	0,000119
Corpul interior	13,196	12,105
Corpul exterior	0,225	0,216

Cel mai mare efect se observă asupra corpului interior, care se mișcă mai rapid și trece mai aproape de corpul central. În configurația verificată nu apare nicio coliziune: toate cele 345601 stări păstrează trei corpuri active.

5.3 Eroarea energiei

Energia inițială este $E_0 = -91886372572{,}5$ J. Deoarece fișierul salvează pozițiile, vitezele au fost reconstruite pentru stările interioare din relația

$$\mathbf{v}_n = \frac{\mathbf{r}_{n+1} - \mathbf{r}_{n-1}}{2h}. \tag{29}$$

Pentru schema leapfrog folosită, cu pas fix și fără coliziuni, această relație rezultă din cele două deplasări consecutive. Ea permite calculul energiei din pozițiile salvate, cu erorile de rotunjire ale reprezentării numerice.

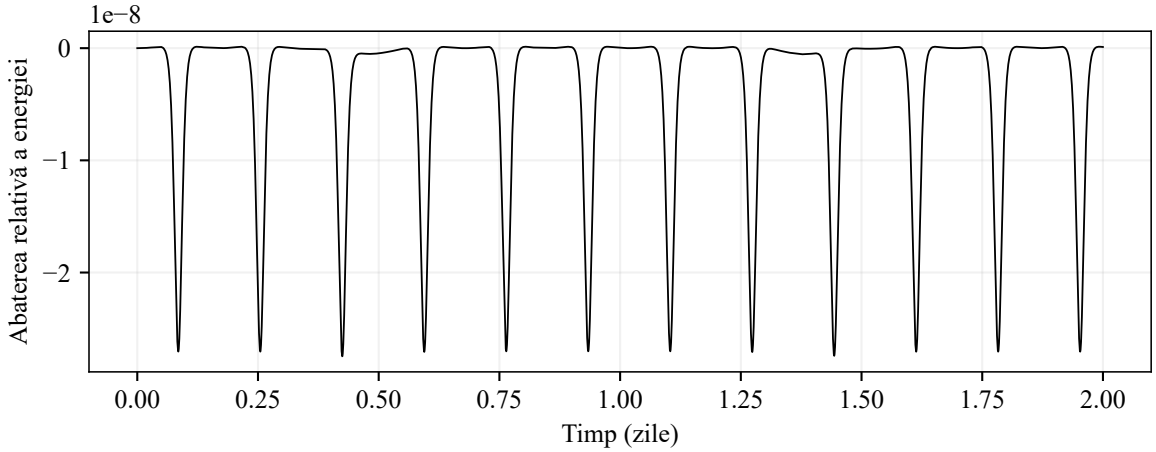


Figura 6: Abaterea relativă a energiei în simularea cu trei corpuri. Graficul folosește energia reconstruită din poziții.

Eroarea absolută maximă pe stările interioare este aproximativ 2524,71 J, iar eroarea relativă este $2{,}748 \times 10^{-8}$, adică aproximativ $0{,}000002748\%$. Energia rămâne apropiată de valoarea inițială, iar comparația de poziție arată separat precizia traiectoriilor pentru acest exemplu.

6 Exemplul 3: coliziunea a patru corpuri

6.1 Configurația simulată

Pentru verificarea coliziunilor s-au folosit patru corpuri identice, așezate simetric în jurul originii. Fiecare are masa 10^{15} kg și raza de 100 m. Vitezele inițiale, de 0,2 m/s, sunt orientate spre centru.

```
4 5000 0.1
1e15 100 -3000 0 0.2 0
1e15 100 3000 0 -0.2 0
1e15 100 0 -3000 0 0.2
1e15 100 0 3000 0 -0.2
```

Simularea durează 5000 s, cu pasul de 0,1 s. Gravitația accelerează apropierea corpurilor. La contact, funcțiile de coliziune le unesc conform relațiilor prezentate în secțiunea 2.4.

6.2 Traiectoriile și momentul unirii

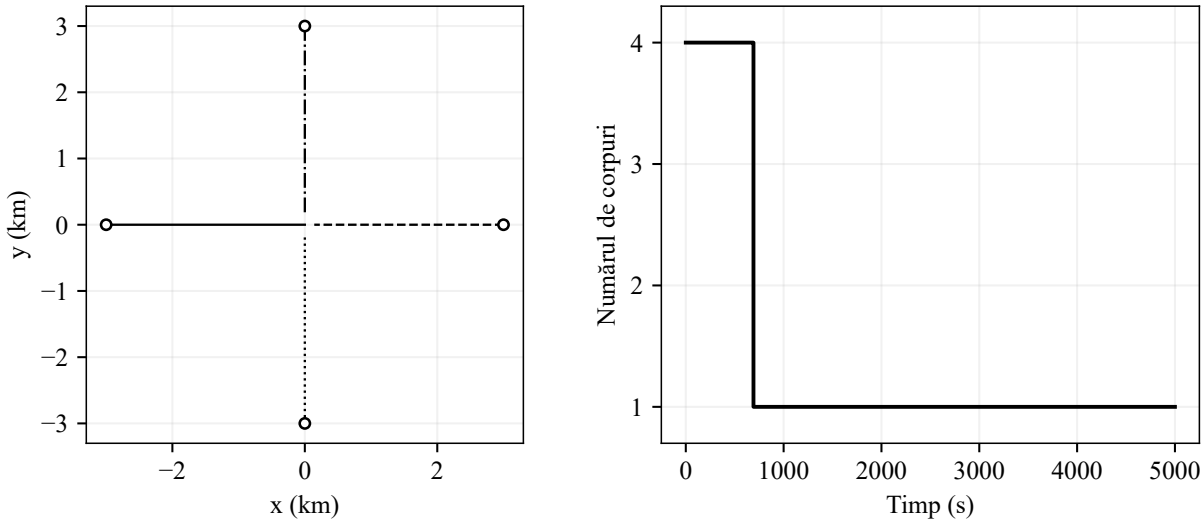


Figura 7: Apropierea celor patru corpuri și schimbarea numărului de corpuri active. Cercurile goale indică pozițiile inițiale. După unire, corpul rezultat rămâne practic în origine.

La aproximativ 691,8 s, numărul de corpuri active scade de la patru la unul. Masa rezultată prin aplicarea regulii de unire este 4×10^{15} kg. Datorită simetriei și impulsului inițial nul, corpul rezultat are o mișcare practic nulă. În rularea refăcută, coordonatele sale finale sunt de ordinul 10^{-12} m.

Acest exemplu verifică detectarea contactului, actualizarea numărului de corpuri și continuarea simulării după unire. El verifică și afișarea unui rezultat în care numărul obiectelor nu mai este constant.

6.3 Interpretarea energiei la coliziune

Înaintea unirii, energia include patru contribuții cinetice și șase perechi de interacțiune gravitațională. După unire, rămâne un singur corp și nu mai există termeni de potențial între corpuri distincte. Energia internă a corpului rezultat nu face parte din model.

Din acest motiv, diferența dintre energia mecanică de dinainte și cea de după coliziune nu este folosită ca eroare de integrare în sensul exemplelor orbitale. Condițiile urmărite aici sunt unirea corpurilor la contact și aplicarea regulilor de conservare a masei și impulsului. Distincția permite interpretarea corectă a celor două tipuri de simulări: orbite fără contact și evoluții cu fuziuni.

7 Concluzii

Programul implementează atracția gravitațională dintre perechi, integrarea leapfrog și unirea corpurilor la contact. Formulele fizice sunt folosite direct în funcțiile de calcul al forțelor, de actualizare a stării și de evaluare a energiei.

Comparația cu două corpuri arată cum influențează pasul temporal precizia. Pentru aceeași orbită și aceeași durată, reducerea pasului de la 5 s la 0,1 s scade eroarea maximă a poziției relative de la aproximativ 753 mm la 0,312 mm. Reducerea erorilor de energie și de poziție este în acord cu ordinul doi al metodei.

Exemplul cu trei corpuri extinde verificarea la un sistem cu interacțiuni suplimentare. Comparația cu DOP853 oferă erori de poziție de aproximativ 13,2 mm pentru corpul interior și 0,225 mm pentru cel exterior. Exemplul cu patru corpuri arată funcționarea unirii la contact și continuarea evoluției cu un singur corp. Împreună, aceste rezultate documentează atât calculul orbital, cât și tratarea coliziunilor.

Bibliografie

[1] E. Hairer, C. Lubich, G. Wanner, *Geometric numerical integration illustrated by the Störmer/Verlet method*, Acta Numerica 12 (2003), 399–450. `https://www.unige.ch/~hairer/preprints/gniverlet.html`.

[2] Jet Propulsion Laboratory, *Fundamentals of Orbital Mechanics*, capitolul 7: ecuația Kepler și calculul poziției pe orbită. `https://spsweb.fltops.jpl.nasa.gov/portaldataops/mpg/MPG_Docs/MPG%20Book/Release/Chapter7-OrbitalMechanics.pdf`.

[3] SciPy, documentația funcției `solve_ivp`, metoda DOP853. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html`. Analiza a folosit SciPy 1.15.3.

A Codul programului principal

Fișierele următoare reproduc sursele documentate. Fragmentele din text explică operațiile principale; anexele păstrează implementarea integrală.

```
//g++ *.cpp -o main && ./main

#include "star.h"

#include <iomanip>


int main()
{
    const char* output = "simulation.bin";
    const char* input = "input.txt";

    ifstream in(input);

    std::cout << setprecision(17);

    int number_of_celestial_objects;
    double max_time, time_step;

    if (!(in >> number_of_celestial_objects >> max_time >> time_step)
        || number_of_celestial_objects <= 0
        || max_time < 0
        || time_step <= 0)
    {
        cerr << "invalid simulation settings\n";
        return 1;
    }

    ofstream out(output, ios::binary | ios::trunc);

    if (!out)
    {
        cerr << "could not open " << output << endl;
        return 1;
    }

    out.write(
        reinterpret_cast<const char*>(&number_of_celestial_objects),
        sizeof(number_of_celestial_objects)
    );

    out.write(
        reinterpret_cast<const char*>(&max_time),
        sizeof(max_time)
    );

    out.write(
        reinterpret_cast<const char*>(&time_step),
        sizeof(time_step)
    );

    celestial_object* celestial_objects =
        new celestial_object[number_of_celestial_objects];

    for (int i = 0; i < number_of_celestial_objects; i++)
    {
        if (!(in >> celestial_objects[i]))
        {
            cerr << "could not read object " << i << '\n';
            delete[] celestial_objects;
            return 1;
        }
    }

    in.close();

    vector_2d* forces = new vector_2d[number_of_celestial_objects];

    const double initial_energy =
```

```
70          total_energy(celestial_objects, number_of_celestial_objects);
71
72      double max_energy = initial_energy;
73      double max_error = 0.0;
74
75      for (double time = 0; time <= max_time; time += time_step)
76      {
77          set_force(
78              forces, celestial_objects, number_of_celestial_objects
79          );
80
81          double current_energy =
82              total_energy(celestial_objects, number_of_celestial_objects);
83
84          if (current_energy > max_energy)
85              max_energy = current_energy;
86
87          if (abs(current_energy - initial_energy) > max_error)
88              max_error = abs(current_energy - initial_energy);
89
90          // Save time and current body count.
91          out.write(
92              reinterpret_cast<const char*>(&time),
93              sizeof(time)
94          );
95
96          out.write(
97              reinterpret_cast<const char*>(&number_of_celestial_objects),
98              sizeof(number_of_celestial_objects)
99          );
100
101         // Save positions before updating any bodies.
102         for (int j = 0; j < number_of_celestial_objects; j++)
103         {
104             const vector_2d position =
105                 celestial_objects[j].get_position();
106
107             out.write(
108                 reinterpret_cast<const char*>(&position.x),
109                 sizeof(double)
110             );
111
112             out.write(
113                 reinterpret_cast<const char*>(&position.y),
114                 sizeof(double)
115             );
116         }
117
118         if (!out)
119         {
120             cerr << "could not write simulation step\n";
121             delete[] celestial_objects;
122             delete[] forces;
123             return 1;
124         }
125
126         // First velocity half-step.
127         for (int j = 0; j < number_of_celestial_objects; j++)
128         {
129             update_velocity(
130                 forces[j], celestial_objects[j], time_step / 2
131             );
132         }
133
134         // Position full-step.
135         for (int j = 0; j < number_of_celestial_objects; j++)
136         {
137             update_position(celestial_objects[j], time_step);
138         }
139
140         set_force(
141             forces, celestial_objects, number_of_celestial_objects
142         );
143
144         // Second velocity half-step.
```

```
145         for (int j = 0; j < number_of_celestial_objects; j++)
146         {
147             update_velocity(
148                 forces[j], celestial_objects[j], time_step / 2
149             );
150         }
151 
152         if ((int)time % 100000 == 0)
153         {
154             std::cout << 100 * time / max_time
155                       << "% completed" << endl;
156         }
157     }
158 
159     delete[] celestial_objects;
160     delete[] forces;
161 
162     out.close();
163 
164     if (!out)
165     {
166         cerr << "could not finish writing output\n";
167         return 1;
168     }
169 
170     std::cout << "initial energy: " << initial_energy
171               << " maximum energy: " << max_energy << endl;
172 
173     std::cout << "energy error: "
174               << abs(initial_energy - max_energy) << endl;
175 
176     std::cout << "max energy error: " << max_error << endl;
177 
178     return 1;
179 }
```

B Definițiile claselor: star.h

```
#include <iostream>
#include <fstream>
#include <cmath>

#define G 6.67430e-11

using namespace std;

class vector_2d
{

    public:

    double x = 0;
    double y = 0;

    vector_2d() = default;

    vector_2d(double x, double y);

    //getters

    //calcul operators
    vector_2d operator+(const vector_2d& AUX) const;
    vector_2d operator*(const vector_2d& AUX) const;
    vector_2d operator*(const double AUX) const;

    //IO operators
    friend istream& operator>>(istream& input, vector_2d& vector);
    friend ostream& operator<<(ostream& output, const vector_2d& vector);

};

class celestial_object
{

    private:

    //mass, radius, position, velocity, acceleration:

    double mass = 0; //kg
    double radius = 0; //m

    vector_2d position{0, 0}; //m
    vector_2d velocity{0, 0}; //m/s
    vector_2d acceleration{0, 0}; //m/s^2

    public:

    celestial_object() = default;

    celestial_object(double mass, double radius, double px, double py, double vx, double vy);

    //getters
    double get_mass() const;
    double get_radius() const;

    vector_2d get_position() const;
    vector_2d get_velocity() const;
    vector_2d get_acceleration() const;

    //setters
    void set_mass(const double& AUX);
    void set_radius(const double& AUX);

    void set_position(const vector_2d& AUX);
    void set_velocity(const vector_2d& AUX);
    void set_acceleration(const vector_2d& AUX);

    //calcul operators

    celestial_object operator+(const celestial_object& AUX) const;
```

```
73
74      //IO operators
75      friend istream& operator>>(istream& input, celestial_object& AUX);
76
77  };
78
79  class force_ij
80  {
81
82      private:
83
84          vector_2d component{0, 0};
85
86          double modulus = 0;
87          double distance = 0;
88
89      public:
90
91      force_ij() = default;
92
93      //getters
94
95      vector_2d get_component();
96
97      double get_modulus();
98      double get_distance();
99
100     //setters
101
102     void set_component(const vector_2d& AUX);
103
104     void set_modulus(const double& AUX);
105     void set_distance(const double& AUX);
106
107 };
108
109 void merge_celestial_objects( celestial_object* objects, int* count, int A, int B);
110
111 void resolve_collisions(celestial_object* objects, int& count);
112
113 void set_force( vector_2d* forces, celestial_object* objects, int& count);
114
115 void update_velocity(const vector_2d AUX, celestial_object& object, double time_step);
116
117 void update_position(celestial_object& object, double time_step);
118
119 double total_energy(const celestial_object* objects, int count);
120
121 int write(double time, const celestial_object& A, ostream& output);
```

C Implementarea modelului: star.cpp

```cpp
#include "star.h"

vector_2d :: vector_2d(double x, double y)
{

    this->x = x;
    this->y = y;

}

vector_2d vector_2d :: operator+(const vector_2d& AUX) const { return vector_2d(x + AUX.x, y + AUX.y); }
vector_2d vector_2d :: operator*(const vector_2d& AUX) const { return vector_2d(x * AUX.x, y * AUX.y); }
vector_2d vector_2d :: operator*(const double AUX) const  { return vector_2d(x * AUX, y * AUX); }

//IO operators
istream& operator>>(istream& input, vector_2d& vector){ input >> vector.x >> vector.y; return input; }
ostream& operator<<(ostream& output, const vector_2d& vector){ output << vector.x << " " << vector.y; return
     output; }

celestial_object :: celestial_object(double mass, double radius, double px, double py, double vx, double vy)
{

    this->mass = mass;
    this->radius = radius;

    position.x = px; position.y = py;
    velocity.x = vx; velocity.y = vy;

}

//getters

double celestial_object :: get_mass() const { return mass; }
double celestial_object :: get_radius() const { return radius; }

vector_2d celestial_object :: get_position() const { return position; }
vector_2d celestial_object :: get_velocity() const { return velocity; }
vector_2d celestial_object :: get_acceleration() const { return acceleration; }

//setters

void celestial_object :: set_mass(const double& AUX) { mass = AUX; }
void celestial_object :: set_radius(const double& AUX) { radius = AUX; }

void celestial_object :: set_position(const vector_2d& AUX) { position = AUX; }
void celestial_object :: set_velocity(const vector_2d& AUX) { velocity = AUX; }
void celestial_object :: set_acceleration(const vector_2d& AUX) { acceleration = AUX; }

celestial_object celestial_object :: operator+(const celestial_object& AUX) const
{

    return celestial_object(mass + AUX.mass, radius + AUX.radius,
                            position.x + AUX.position.x, position.y + AUX.position.y,
                            velocity.x + AUX.velocity.x, velocity.y + AUX.velocity.y);
}

istream& operator>>(istream& input, celestial_object& AUX)
{

    input >> AUX.mass >> AUX.radius

    >> AUX.position.x >> AUX.position.y
    >> AUX.velocity.x >> AUX.velocity.y;

    AUX.set_acceleration(vector_2d(0, 0));

    return input;

}

vector_2d force_ij :: get_component(){ return vector_2d(component.x, component.y); }
double force_ij :: get_modulus(){ return modulus; }
```

```cpp
double force_ij :: get_distance(){ return distance; }

void force_ij :: set_component(const vector_2d& AUX) { component = AUX; }
void force_ij :: set_modulus(const double& AUX) { modulus = AUX; }
void force_ij :: set_distance(const double& AUX) { distance = AUX; }

void merge_celestial_objects( celestial_object* objects, int* count, int A, int B)
{

    const double ma = objects[A].get_mass();
    const double mb = objects[B].get_mass();
    const double mass = ma + mb;

    const vector_2d position =
        (objects[A].get_position() * ma
       + objects[B].get_position() * mb) * (1.0 / mass);

    const vector_2d velocity =
        (objects[A].get_velocity() * ma
       + objects[B].get_velocity() * mb) * (1.0 / mass);

    const double ra = objects[A].get_radius();
    const double rb = objects[B].get_radius();

    const double radius =
        std::cbrt(ra * ra * ra + rb * rb * rb);

    objects[A] = celestial_object(
        mass, radius,
        position.x, position.y,
        velocity.x, velocity.y
    );

    for (int k = B; k < *count - 1; ++k)
        objects[k] = objects[k + 1];

    --(*count);
}

void resolve_collisions(celestial_object* objects, int& count)
{
    bool merged;

    do
    {
        merged = false;

        for (int i = 0; i < count - 1 && !merged; ++i)
        {
            for (int j = i + 1; j < count; ++j)
            {
                const vector_2d a = objects[i].get_position();
                const vector_2d b = objects[j].get_position();

                const double distance =
                    std::hypot(b.x - a.x, b.y - a.y);

                const double total_radius =
                    objects[i].get_radius() + objects[j].get_radius();

                if (distance <= total_radius)
                {
                    merge_celestial_objects(objects, &count, i, j);
                    merged = true;
                    break;
                }
            }
        }
    }
    while (merged);
}

void set_force( vector_2d* forces, celestial_object* objects, int& count)
{
    resolve_collisions(objects, count);
```

```
147
148     for (int i = 0; i < count; ++i)
149         forces[i] = vector_2d(0, 0);
150
151     for (int i = 0; i < count - 1; ++i)
152     {
153         for (int j = i + 1; j < count; ++j)
154         {
155             const vector_2d a = objects[i].get_position();
156             const vector_2d b = objects[j].get_position();
157
158             const vector_2d displacement(b.x - a.x, b.y - a.y);
159             const double distance =
160                 std::hypot(displacement.x, displacement.y);
161
162             if (distance == 0.0)
163                 continue;
164
165             const double magnitude =
166                 G * objects[i].get_mass() * objects[j].get_mass()
167                 / distance / distance;
168
169             const vector_2d force =
170                 displacement * (magnitude / distance);
171
172             forces[i] = forces[i] + force;
173             forces[j] = forces[j] + force * (-1.0);
174         }
175     }
176 }
177
178 void update_velocity(const vector_2d AUX, celestial_object& object, double time_step)
179 {
180
181     object.set_acceleration(vector_2d(AUX.x / object.get_mass(), AUX.y / object.get_mass()));
182
183     object.set_velocity(vector_2d(object.get_velocity().x + object.get_acceleration().x * time_step,
184                                   object.get_velocity().y + object.get_acceleration().y * time_step));
185
186 }
187
188 void update_position(celestial_object& object, double time_step)
189 {
190
191     object.set_position(vector_2d(object.get_position().x + object.get_velocity().x * time_step,
192                                   object.get_position().y + object.get_velocity().y * time_step));
193
194 }
195
196 double total_energy(const celestial_object* objects, int count)
197 {
198     double energy = 0.0;
199
200     for (int i = 0; i < count; ++i)
201     {
202         const vector_2d velocity = objects[i].get_velocity();
203
204         energy += 0.5 * objects[i].get_mass()
205                 * (velocity.x * velocity.x + velocity.y * velocity.y);
206
207         for (int j = i + 1; j < count; ++j)
208         {
209             const vector_2d a = objects[i].get_position();
210             const vector_2d b = objects[j].get_position();
211
212             const double distance = std::hypot(b.x - a.x, b.y - a.y);
213
214             // Requires distinct centers: distance > 0.
215             energy -= G * objects[i].get_mass()
216                         * objects[j].get_mass() / distance;
217         }
218     }
219
220     return energy;
221 }
```

```
222
223 int write(double time, const celestial_object& A, ostream& output)
224 {
225     static_assert(
226         is_trivially_copyable_v<celestial_object>,
227         "object requires custom serialization"
228     );
229
230     output.write(reinterpret_cast<const char*>(&time), sizeof(time));
231
232     const vector_2d position = A.get_position();
233
234     output.write(reinterpret_cast<const char*>(&position), sizeof(position));
235
236     if (!output)
237     {
238         std::cerr << "could not write simulation record\n";
239         return 1;
240     }
241
242     return 0;
243 }
```

D Vizualizatorul: graph.py

```
import argparse
import struct
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Button, Slider, TextBox


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('file', nargs='?', default='simulation.bin')
    parser.add_argument('--save', help='Save a static image instead of opening the viewer')
    args = parser.parse_args()
    path = Path(args.file)
    with path.open('rb') as stream:
        header = stream.read(20)
    if len(header) != 20:
        parser.error('Incomplete header')
    count, duration, step = struct.unpack('<idd', header)
    if count < 1 or not np.isfinite([duration, step]).all() or step <= 0:
        parser.error('Invalid header')
    frames_data = []
    counts = []
    with path.open('rb') as stream:
        stream.seek(20)
        previous = -np.inf
        while True:
            frame_header = stream.read(12)
            if not frame_header:
                break
            if len(frame_header) != 12:
                parser.error('Incomplete step header')
            time, active = struct.unpack('<di', frame_header)
            if not np.isfinite(time) or time <= previous or not 1 <= active <= count:
                parser.error('Invalid step; regenerate the binary with the new output block')
            if counts and active > counts[-1]:
                parser.error('Body count increased unexpectedly')
            payload = stream.read(active * 16)
            if len(payload) != active * 16:
                parser.error('Incomplete positions')
            xy = np.frombuffer(payload, dtype='<f8').reshape(active, 2)
            if not np.isfinite(xy).all():
                parser.error('Nonfinite positions')
            row = np.full((count, 3), np.nan)
            row[:, 0] = time
            row[:active, 1:3] = xy
            frames_data.append(row)
            counts.append(active)
            previous = time
    if not frames_data:
        parser.error('No saved steps')
    records = np.asarray(frames_data)
    del frames_data
    counts = np.asarray(counts)
    frames = len(records)
    times = records[:, 0, 0]
    low = np.nanmin(records[:, :, 1:3], axis=(0, 1))
    high = np.nanmax(records[:, :, 1:3], axis=(0, 1))
    changes = np.flatnonzero(np.r_[False, counts[1:] != counts[:-1]])

    fig, ax = plt.subplots(figsize=(10, 8))
    fig.subplots_adjust(bottom=.25, top=.9)
    center = (low + high) / 2
    span = max(float((high - low).max()), 1.0)
    ax.set(xlim=(center[0]-.55*span, center[0]+.55*span),
           ylim=(center[1]-.55*span, center[1]+.55*span),
           xlabel='X (meters)', ylabel='Y (meters)')
    ax.set_aspect('equal')
    ax.grid(True, alpha=.25)
    trails, markers = [], []
    for body in range(count):
        line, = ax.plot([], [], lw=1, label=f'Body {body}')
```

```
        marker, = ax.plot([], [], 'o', color=line.get_color(), markersize=7)
        trails.append(line)
        markers.append(marker)
    ax.legend()
    title = ax.set_title('')
    status = fig.text(.12, .02, '', fontsize=9)
    state = {'index': 0, 'playing': False, 'updating': False}
    slider = Slider(fig.add_axes([.16, .15, .69, .03]), 'Time (s)',
                    float(times[0]), float(times[-1]) if frames > 1 else float(times[0])+1,
                    valinit=float(times[0]))
    button = Button(fig.add_axes([.12, .065, .14, .05]), 'Play')
    back = Button(fig.add_axes([.28, .065, .10, .05]), 'Previous')
    forward = Button(fig.add_axes([.40, .065, .10, .05]), 'Next')
    box = TextBox(fig.add_axes([.68, .065, .18, .05]), 'Jump to (s) ', initial='0')

    def pause():
        state['playing'] = False
        button.label.set_text('Play')

    def nearest(value):
        k = int(np.searchsorted(times, value))
        if k == 0:
            return 0
        if k >= frames:
            return frames - 1
        return k if times[k]-value < value-times[k-1] else k-1

    def show(index):
        state['index'] = int(index)
        # Limit trail rendering, but always show the exact selected saved state.
        # Indices may change after merging. Start new trails after each count change.
        history_start = int(changes[np.searchsorted(changes, index, side='right')-1]) if np.searchsorted(
     changes, index, side='right') else 0
        stride = max(1, (index-history_start+1)//10000)
        indices = np.unique(np.r_[np.arange(history_start, index+1, stride), index])
        xy = np.asarray(records[indices, :, 1:3])
        for body in range(count):
            trails[body].set_data(xy[:, body, 0], xy[:, body, 1])
            markers[body].set_data([records[index, body, 1]], [records[index, body, 2]])
        title.set_text(f'Time: {times[index]:,.6f} s | Bodies: {counts[index]} | Step {index+1:,}/{frames:,}')
        state['updating'] = True
        try:
            slider.set_val(float(times[index]))
        finally:
            state['updating'] = False
        fig.canvas.draw_idle()

    def seek(value):
        if state['updating']:
            return
        pause()
        show(nearest(value))
        status.set_text('Paused at nearest saved time. Space: play/pause; arrows: one saved step.')

    def jump(text):
        try:
            value = float(text)
            if not np.isfinite(value):
                raise ValueError
        except ValueError:
            status.set_text('Enter a finite time in seconds.')
            fig.canvas.draw_idle()
            return
        pause()
        show(nearest(value))
        status.set_text(f'Requested {value:g} s; showing nearest saved time {times[state["index"]]:g} s.')
        fig.canvas.draw_idle()

    def toggle(event=None):
        if not state['playing'] and state['index'] == frames-1:
            show(0)
        state['playing'] = not state['playing']
        button.label.set_text('Pause' if state['playing'] else 'Play')
        fig.canvas.draw_idle()

```

```
    def move(amount):
        pause()
        show(max(0, min(frames-1, state['index']+amount)))

    def key(event):
        if event.inaxes is box.ax:
            return
        if event.key == ' ':
            toggle()
        elif event.key == 'right':
            move(1)
        elif event.key == 'left':
            move(-1)

    def tick():
        if state['playing']:
            show(min(frames-1, state['index']+max(1, (frames-1)//1800)))
            if state['index'] == frames-1:
                pause()
                fig.canvas.draw_idle()

    slider.on_changed(seek)
    box.on_submit(jump)
    button.on_clicked(toggle)
    back.on_clicked(lambda event: move(-1))
    forward.on_clicked(lambda event: move(1))
    fig.canvas.mpl_connect('key_press_event', key)
    timer = fig.canvas.new_timer(interval=33)
    timer.add_callback(tick)
    show(0)
    status.set_text('Space: play/pause. Arrows: one saved step. Jump uses nearest saved time.')
    print(f'{frames:,} saved steps, {count} bodies; {times[0]:g} to {times[-1]:g} seconds.')
    if args.save:
        show(frames-1)
        fig.savefig(args.save, dpi=160)
        plt.close(fig)
    else:
        timer.start()
        plt.show()
        timer.stop()


if __name__ == '__main__':
    main()
```

E Programul folosit pentru măsurarea erorilor

Programul următor folosește funcțiile din `star.cpp` și înregistrează energia la fiecare pas. Argumentele sale sunt: fișierul de intrare, ieșirea CSV, intervalul de salvare exprimat în pași și ieșirea JSON. Datele și comparațiile numerice sunt incluse în arhiva sursei LaTeX.

```
#include "src/star.h"
#include <algorithm>
#include <cstdlib>
#include <iomanip>
#include <type_traits>
#include <vector>
int main(int argc, char **argv) {
  if (argc != 5)
    return 2;
  ifstream in(argv[1]);
  int n;
  double T, h;
  in >> n >> T >> h;
  vector<celestial_object> b(n);
  for (auto &x : b)
    in >> x;
  vector<vector_2d> F(n);
  ofstream out(argv[2]);
  out << setprecision(17);
  int stride = atoi(argv[3]);
  double E0 = total_energy(b.data(), n), maxE = E0, maxAbs = 0, minE = E0;
  double minD = 1e300;
  long long k = 0;
  double last = 0;
  out << "t,E";
  for (int i = 0; i < n; i++)
    out << ",x" << i << ",y" << i << ",vx" << i << ",vy" << i;
  out << "\n";
  for (double t = 0; t <= T; t += h, ++k) {
    set_force(F.data(), b.data(), n);
    double E = total_energy(b.data(), n);
    maxE = max(maxE, E);
    minE = min(minE, E);
    maxAbs = max(maxAbs, abs(E - E0));
    last = t;
    for (int i = 0; i < n; i++)
      for (int j = i + 1; j < n; j++) {
        auto x = b[i].get_position(), y = b[j].get_position();
        minD = min(minD, hypot(x.x - y.x, x.y - y.y));
      }
    if (k % stride == 0 || t + h > T) {
      out << t << "," << E;
      for (int i = 0; i < n; i++) {
        auto r = b[i].get_position(), v = b[i].get_velocity();
        out << "," << r.x << "," << r.y << "," << v.x << "," << v.y;
      }
      out << "\n";
    }
    for (int i = 0; i < n; i++)
      update_velocity(F[i], b[i], h / 2);
    for (int i = 0; i < n; i++)
      update_position(b[i], h);
    set_force(F.data(), b.data(), n);
    for (int i = 0; i < n; i++)
      update_velocity(F[i], b[i], h / 2);
  }
  ofstream s(argv[4]);
  s << setprecision(17) << "{\"h\":" << h << ",\"T\":" << T << ",\"E0\":" << E0
    << ",\"Emax\":" << maxE << ",\"Emin\":" << minE << ",\"max_abs\":" << maxAbs
    << ",\"positive\":" << maxE - E0 << ",\"frames\":" << k
    << ",\"last\":" << last << ",\"min_distance\":" << minD
    << ",\"active\":" << n << "}";
}
```